



User-Defined Function in C

Last Updated : 01 Jun, 2023



A **user-defined function** is a type of function in C language that is defined by the user himself to perform some specific task. It provides code reusability and modularity to our program. User-defined functions are different from built-in functions as their working is specified by the user and no header file is required for their usage.

In this article, we will learn about **user-defined function**, **function prototype**, **function definition**, **function call**, and **different ways in which we can pass parameters to a function**.

How to use User-Defined Functions in C?

To use a user-defined function, we first have to understand the different parts of its syntax. The user-defined function in C can be divided into three parts:

1. Function Prototype
2. Function Definition
3. Function Call

C Function Prototype

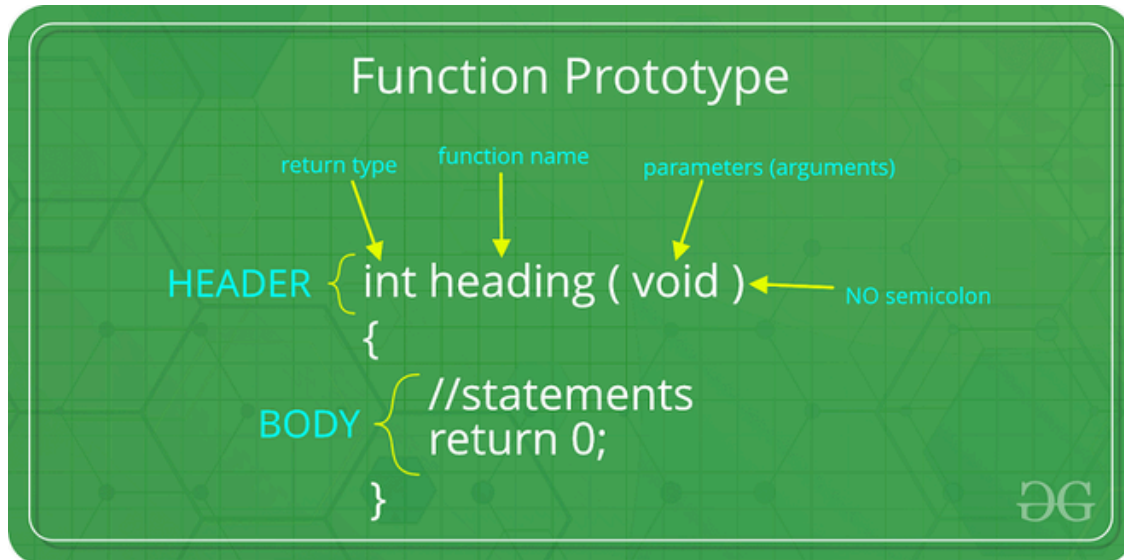
A function prototype is also known as a function declaration which specifies the **function's name**, **function parameters**, and **return type**. The function prototype does not contain the body of the function. It is basically used to inform the compiler about the existence of the user-defined function which can be used in the later part of the program.

Syntax

```
return_type function_name (type1 arg1, type2 arg2, ... typeN argN);
```

We can also skip the name of the arguments in the function prototype. So,

```
return_type function_name (type1 , type2 , ... typeN);
```



C Function Definition

Once the function has been called, the function definition contains the actual statements that will be executed. All the statements of the function definition are enclosed within **{ } braces**.

Syntax

```
return_type function_name (type1 arg1, type2 arg2 .... typeN argN) {

    // actual statements to be executed
    // return value if any
}
```

Note: If the function call is present after the function definition, we can skip the function prototype part and directly define the function.

C Function Call

In order to transfer control to a user-defined function, we need to call it. Functions are called using their names followed by round brackets. Their arguments are passed inside the brackets.

Syntax

```
function_name(arg1, arg2, ... argN);
```

Example of User-Defined Function

The following C program illustrates how to use user-defined functions in our program.

C

```
// C Program to illustrate the use of user-defined function
#include <stdio.h>

// Function prototype
int sum(int, int);

// Function definition
int sum(int x, int y)
{
    int sum;
    sum = x + y;
    return x + y;
}

// Driver code
int main()
{
    int x = 10, y = 11;

    // Function call
    int result = sum(x, y);
    printf("Sum of %d and %d = %d ", x, y, result);

    return 0;
}
```

Output

Sum of 10 and 11 = 21

Components of Function Definition

There are three components of the function definition:

1. Function Parameters
2. Function Body
3. Return Value

1. Function Parameters

Function parameters (also known as arguments) are the values that are passed to the called function by the caller. We can pass none or any number of function parameters to the function.

We have to define the function name and its type in the function definition and we can only pass the same number and type of parameters in the function call.

Example

```
int foo (int a, int b);
```

Here, **a** and **b** are function parameters.

***Note:** C language provides a method using which we can pass variable number of arguments to the function. Such functions are called [variadic function](#).*

2. Function Body

The function body is the set of statements that are enclosed within { } braces. They are the statements that are executed when the function is called.

Example

```
int foo (int a, int b) {  
    int sum = a + b;  
    return sum;  
}
```

Here, the statements between { and } is function body.

3. Return Value

The return value is the value returned by the function to its caller. A function can only return a single value and it is optional. If no value is to be returned, the return type is defined as void.

The **return keyword** is used to return the value from a function.

Syntax

```
return (expression);
```

Example

```
int foo (int a, int b) {  
    return a + b;  
}
```

***Note:** We can use pointers or structures to [return multiple values from a function in C](#).*

Passing Parameters to User-Defined Functions

We can pass parameters to a function in C using two methods:

1. Call by Value
2. Call by Reference

1. Call by value

In call by value, a copy of the value is passed to the function and changes that are made to the function are not reflected back to the values. Actual and formal arguments are created in different memory locations.

Example

C

```
// C program to show use of
```

```
// call by value
#include <stdio.h>

void swap(int a, int b)
{
    int temp = a;
    a = b;
    b = temp;
}

// Driver code
int main()
{
    int x = 10, y = 20;
    printf("Values of x and y before swap are: %d, %d\n", x,
        y);
    swap(x, y);
    printf("Values of x and y after swap are: %d, %d", x,
        y);
    return 0;
}
```

Output

Values of x and y before swap are: 10, 20

Values of x and y after swap are: 10, 20

Note: Values aren't changed in the call by value since they aren't passed by reference.

2. Call by Reference

In a call by Reference, the address of the argument is passed to the function, and changes that are made to the function are reflected back to the values. We use the pointers of the required type to receive the address in the function.

Example

C

```
// C program to implement
// Call by Reference
#include <stdio.h>

void swap(int* a, int* b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Driver code
int main()
{
    int x = 10, y = 20;
    printf("Values of x and y before swap are: %d, %d\n", x,
        y);
    swap(&x, &y);
    printf("Values of x and y after swap are: %d, %d", x,
        y);
    return 0;
}
```

Output

Values of x and y before swap are: 10, 20

Values of x and y after swap are: 20, 10

For more details, refer to this article – [Difference between Call by Value and Call by Reference](#)

Advantages of User-Defined Functions

The advantages of using functions in the program are as follows:

- One can avoid duplication of code in the programs by using functions. Code can be written more quickly and be more readable as a result.
- Code can be divided and conquered using functions. This process is known as Divide and Conquer. It is difficult to write large amounts of code within the main function, as well as testing and debugging. Our one task can be divided into several smaller sub-tasks by using functions, thus reducing the overall complexity.

- For example, when using pow, sqrt, etc. in C without knowing how it is implemented, one can hide implementation details with functions.
- With little to no modifications, functions developed in one program can be used in another, reducing the development time.

Summer-time is here and so is the time to skill-up! More than 5,000 learners have now completed their journey from **basics of DSA to advanced level development programs** such as Full-Stack, Backend Development, Data Science.

And why go anywhere else when our [DSA to Development: Coding Guide](#) will help you master all this in a few months! Apply now to our [DSA to Development Program](#) and our counsellors will connect with you for further guidance & support.



kaml... [+ Follow](#)

24

Previous Article

C Functions

Next Article

Parameter Passing Techniques in C

Similar Reads

Function Interposition in C with an example of user defined malloc()

Function interposition is the concept of replacing calls to functions in dynamic libraries with calls to user-defined wrappers. What are applications? We can...

3 min read

Types of User Defined Functions in C

A user-defined function is one that is defined by the user when writing any program, as we do not have library functions that have predefined definitions. T...

4 min read

How Linkers Resolve Global Symbols Defined at Multiple Places?

At compile time, the compiler exports each global symbol to the assembler as either strong or weak, and the assembler encodes this information implicitly in...

4 min read

How to input or read a Character, Word and a Sentence from user in C?

C is a procedural programming language. It was initially developed by Dennis Ritchie as a system programming language to write an operating system. The...

4 min read

C Function Arguments and Function Return Values

Prerequisite: Functions in C A function in C can be called either with arguments or without arguments. These functions may or may not return values to the calling...

6 min read

[View More Articles](#)

Article Tags :

[C Language](#)



Sanchhaya Education Private Limited

A-143, 9th Floor, Sovereign Corporate Tower, Sector-136, Noida, Uttar Pradesh - 201305



Company

About Us
Legal
Careers
In Media
Contact Us
Advertise with us
GFG Corporate Solution
Placement Training Program

Explore

Job-A-Thon Hiring Challenge
Hack-A-Thon
GfG Weekly Contest
Offline Classes (Delhi/NCR)
DSA in JAVA/C++
Master System Design
Master CP
GeeksforGeeks Videos

Languages

Python
Java
C++
PHP
GoLang
SQL
R Language
Android Tutorial

Data Science & ML

Data Science With Python
Data Science For Beginner
Machine Learning Tutorial
ML Maths
Data Visualisation Tutorial
Pandas Tutorial
NumPy Tutorial
NLP Tutorial
Deep Learning Tutorial

Python Tutorial

Python Programming Examples
Django Tutorial
Python Projects
Python Tkinter
Web Scraping
OpenCV Tutorial
Python Interview Question

DevOps

Git
AWS
Docker
Kubernetes
Azure
GCP
DevOps Roadmap

School Subjects

Mathematics
Physics
Chemistry
Biology
Social Science
English Grammar

DSA

Data Structures
Algorithms
DSA for Beginners
Basic DSA Problems
DSA Roadmap
DSA Interview Questions
Competitive Programming

Web Technologies

HTML
CSS
JavaScript
TypeScript
ReactJS
NextJS
NodeJs
Bootstrap
Tailwind CSS

Computer Science

GATE CS Notes
Operating Systems
Computer Network
Database Management System
Software Engineering
Digital Logic Design
Engineering Maths

System Design

High Level Design
Low Level Design
UML Diagrams
Interview Guide
Design Patterns
OOAD
System Design Bootcamp
Interview Questions

Commerce

Accountancy
Business Studies
Economics
Management
HR Management
Finance

[Income Tax](#)

Databases

[SQL](#)
[MYSQL](#)
[PostgreSQL](#)
[PL/SQL](#)
[MongoDB](#)

Preparation Corner

[Company-Wise Recruitment Process](#)
[Resume Templates](#)
[Aptitude Preparation](#)
[Puzzles](#)
[Company-Wise Preparation](#)
[Companies](#)
[Colleges](#)

Competitive Exams

[JEE Advanced](#)
[UGC NET](#)
[UPSC](#)
[SSC CGL](#)
[SBI PO](#)
[SBI Clerk](#)
[IBPS PO](#)
[IBPS Clerk](#)

More Tutorials

[Software Development](#)
[Software Testing](#)
[Product Management](#)
[Project Management](#)
[Linux](#)
[Excel](#)
[All Cheat Sheets](#)

Free Online Tools

[Typing Test](#)
[Image Editor](#)
[Code Formatters](#)
[Code Converters](#)
[Currency Converter](#)
[Random Number Generator](#)
[Random Password Generator](#)

Write & Earn

[Write an Article](#)
[Improve an Article](#)
[Pick Topics to Write](#)
[Share your Experiences](#)
[Internships](#)

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved